APCCAS 2026 Fukuoka JAPAN

SAGE-16: A 16-DSP Reconfigurable Fabric for Edge Vision and Attitude Kernels

Poster NO.2391

Harshvardhan Mishra[1], Madhav Rao[1]

1. Department of Electronics and Communication Engineering, IIIT Bangalore, India

One shared 4×4 MAC fabric for camera and IMU workloads

Nano-drones, wearables and robotic end-effectors must run CNN perception and quaternion-based attitude estimation within one tight latency, power and area budget [1], [2]. Existing work optimizes one side at a time: MCU software stacks [5], [6], CNN accelerators [7]–[16], [22], CGRA-style fabrics [23]–[25], dedicated quaternion units [17], [18], or conv/GEMM engines reconfigured over a bus [19], [20].

SAGE-16 time-multiplexes all three kernel families on one array of 16 DSP48E1 slices and switches between them in a single clock cycle, without a bitstream reload.

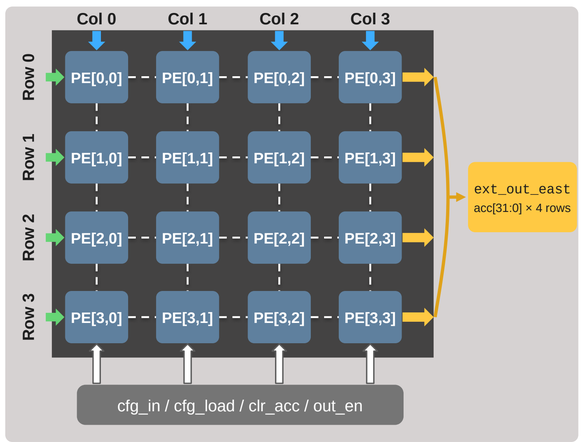


Fig. 1. The shared 4×4 fabric. Row operands enter on `in_bypass` (green) and column operands on `in_b_col` (blue); dashed links form the 16-bit N/S/E/W mesh.

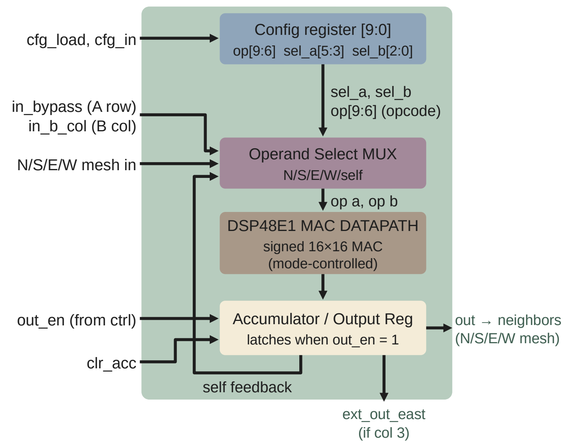


Fig. 2. Inside one PE: the config word selects the operands, a DSP48E1 performs the signed 16×16 MAC, and `clr_acc` / `out_en` control the 32-bit accumulator.

−66.7% — DSP slices: 16 rather than 48 for three cores

1 cycle — kernel reconfiguration: no reset, no bitstream reload

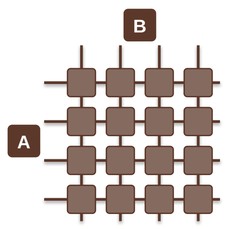
1.436 — GMAC/s on conv 3×3: post-route at 159.6 MHz

38.6× — fewer cycles than an M4: conv 3×3 tile, scalar baseline

49.09 M — bit-exact RTL checks: full MNIST set, zero mismatches

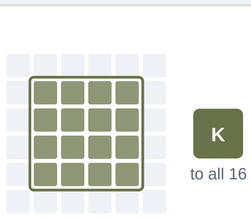
How operands move through the array

Each PE holds one DSP48E1 and a 32-bit accumulator; the array is output-stationary. Operands arrive over a row broadcast and a column broadcast, and all 16 results are read out in parallel.



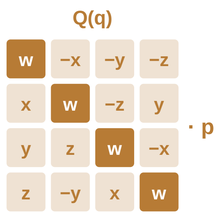
Matmul mode 00

At step k, row i receives A[i][k] and column j receives B[k][j], so every PE accumulates one product. After four steps the array holds C = A·B.



Conv 3×3 mode 01

Each PE takes its own pixel from a 6×6 input patch over a per-PE path, while the current weight is broadcast to all sixteen. Nine taps produce a full 4×4 output tile.

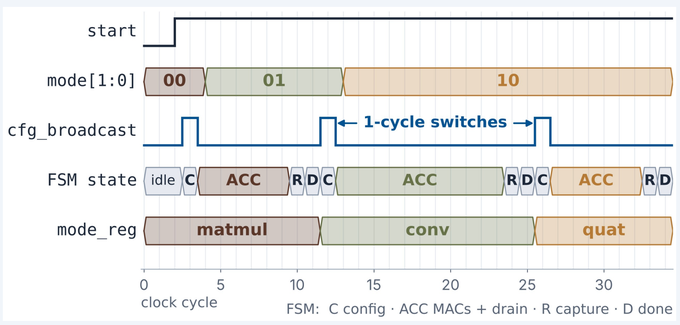


Quaternion mode 10

A unit quaternion encodes a 3-D orientation, and IMU filters such as Madgwick [3] and Mahony [4] update it with Hamilton products. Since q ⊗ p equals Q(q)·p, the product reuses the matmul datapath, four products per pass.

Switching kernels at run time

00	**Matmul, unsigned** 4×4 block, 64 MACs	11 cycles
01	**Conv 3×3** 4×4 output tile, 144 MACs	16 cycles
10	**Quaternion batch** four Hamilton products, 64 MACs	11 cycles
11	**Matmul, signed** used by the CNNs' fully connected layers	11 cycles



Simulated sage16_top running matmul, conv and quaternion back to back: `mode_reg` changes on one `cfg_broadcast` cycle. All 6 ordered transitions were tested and all 192 outputs were bit-exact. The host connects through MMIO or a PicoRV32 PCPI wrapper [26].

Three independent lines of evidence

RTL — Every output checked against a bit-exact integer model: 16 639 regression checks and 49.09 M on MNIST, no failures.

FPGA — Vivado 2025.1, post-route, out-of-context, Artix-7 xc7a35t.

MCU — STM32G491 Cortex-M4 at 170 MHz on the same integer tensors, scalar C and CMSIS-DSP `SMLAD`.

What sharing the array saves

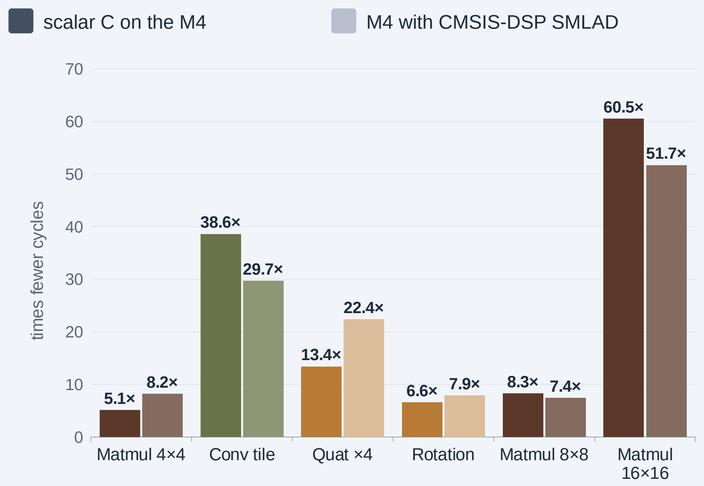


−66.7% DSP slices, 6.8% fewer LUTs

Design	LUT	FF	DSP	Fmax
SAGE-16 core	**4879**	**1606**	**16**	**159.6**
with MMIO	5200	2896	16	162.5
with PCPI	5233	2382	16	156.4

Post-route, Fmax in MHz, no BRAM. Baseline: the three kernels as separate 16-DSP designs on the same device.

Cycle counts against a Cortex-M4



Clock-cycle ratios, median of 100 runs; operand transfer excluded, tiled cases accumulate on the host.

Where the cycles go



Only 4 of 11 cycles (9 of 16 for conv) issue MACs; per-call setup and pipeline drain account for the rest.

Complete models, MACs on the fabric

87.69% top-1, fixed point

CIFAR-10, VGG-style CNN

91.84% in floating point. A 64-image RTL run reproduced the exported predictions exactly, and the 1.1 MB model would not even fit in the M4's flash.

101.2 µs per image, about 22× fewer cycles than the M4

MNIST, compact INT4 CNN

95.04% top-1 from our own INT4 model of about 10 k weights. The fabric reproduces all 10 000 images bit-exactly, so the accuracy is the model's, not the hardware's.

13.4× fewer cycles than scalar M4

Quaternion attitude

A batch of four Hamilton products completes in 11 cycles, and a full vector rotation in 22.



The first ten MNIST test images with the labels the fabric predicted. The 5 read as a 9 is one of the 4.96% the model gets wrong.

One MNIST inference plus an attitude update takes about 101.3 µs (≈9.9 kHz), well above the 30–60 fps vision and ~1 kHz IMU rates of nano-drones [1], [2].

Relation to other FPGA accelerators

Work	Device	DSP	Metric	Kernels	Switch
Yan'22 [27]	A7 35T	17	340.7 µs/img	CNN	no
Eresh'25 [19]	A7 35T	n/r	1.6 Gb/s peak	conv, GEMM	AXI
Shi'23 [14]	KV260	1248	75.19 GOPS	CNN	reload
Shen'25 [15]	Zynq 7020	220	n/r	conv, DW	no
Yang'23 [16]	Zynq 7020	113	29.53 GOPS	CNN	no
SAGE-16	**A7 35T**	**16**	**1.436 GMAC/s**	**all three**	**1 cycle**

Each work reports its own metric, so the table positions SAGE-16 rather than ranking it. Vivado's vectorless estimate puts the core at 0.188 W, or 7.64 GMAC/s per watt on conv 3×3.

What to take away

- A single 16-DSP array executes conv, matmul and quaternion kernels and reconfigures between them in one cycle.
- It needs a third of the DSP slices of three dedicated accelerators on the same device.
- Matmul, conv and quaternion kernels need 5.1× to 38.6× fewer cycles than scalar code on a Cortex-M4.

Next: hardware pooling, joint camera and IMU scheduling, measured board power, and an ASIC.

code and testbenches [21]

REFERENCES

[1] D. Palossi et al., IEEE IoT J., 6(5), 2019.
[2] A. Suleiman et al., IEEE JSSC, 54(4), 2019.
[3] S. O. H. Madgwick, Univ. of Bristol, Tech. Rep., 2010.
[4] R. Mahony et al., IEEE TAC, 53(5), 2008.
[5] L. Lai et al., CMSIS-NN, arXiv:1801.06601, 2018.
[6] A. Garofalo et al., PULP-NN, Phil. Trans. A, 2020.
[7] Y.-H. Chen et al., Eyeriss, ISSCC, 2016.
[8] Y.-H. Chen et al., Eyeriss v2, IEEE JETCAS, 9(2), 2019.
[9] Y. Umuroglu et al., FINN, FPGA, 2017.
[10] M. Blott et al., FINN-R, ACM TRETS, 11(3), 2018.
[11] M. P. Véstias et al., IEEE Access, 8, 2020.
[12] K. M. V. Gowda et al., Electronics, 11(10), 2022.
[13] N. P. Jouppi et al., TPU, ISCA, 2017.
[14] K. Shi et al., Information, 14(3), 2023.
[15] J. Shen et al., AI, 6(10), 2025.
[16] R. Yang et al., Sensors, 23(14), 2023.
[17] A. Cariow et al., Applied Sciences, 14(18), 2024.
[18] Y. Zhu et al., Sensors, 22(17), 2022.
[19] Eresh and S. K. B., CSITSS, 2025.
[20] Swati et al., IEEE iSES, 2023.
[21] SAGE-16 RTL, github.com/hvm007/sage_16, 2026.
[22] M. P. Véstias et al., Lite-CNN, FPL, 2018.
[23] N. Carpentieri et al., OpenEdgeCGRA, CF, 2024.
[24] M. Karunaratne et al., HyCUBE, DAC, 2017.
[25] J. Lee and J. Lee, IEEE Trans. CAD, 41(10), 2022.
[26] C. Wolf, PicoRV32, GitHub, 2015.
[27] F. Yan et al., Sensors, 22(16), 2022.

Harshvardhan Mishra Harshvardhan.Mishra@iiitb.ac.in **Madhav Rao** mr@iiitb.ac.in

Department of Electronics and Communication Engineering, IIIT Bangalore, India